

LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: K. Sirisha

Roll No.: A24126552260

Section: B

EXPERIMENT 4 - DOCUMENT OBJECT MODEL (DOM) AND DYNAMIC MANIPULATION (CO-1)

1. TITLE

Explain the Document Object Model (DOM) and Demonstrate Dynamic Modification of the Content, Style, and Attributes of HTML Elements Using JavaScript.

2. AIM

To study the structure of the Document Object Model and to design a webpage that demonstrates how JavaScript dynamically changes the content, the style, and the attributes of HTML elements at run time without reloading the page.

3. OBJECTIVE

- To understand the tree structure of the DOM and the relationship between nodes.
- To select elements from the DOM using `getElementById()`, `getElementsByClassName()`, `querySelector()` and `querySelectorAll()`.
- To modify the content of elements using `textContent`, `innerHTML`, and `createElement()` with `appendChild()`.
- To modify the appearance of elements using the `style` property and the `classList` API.
- To read, modify, and delete element attributes using `getAttribute()`, `setAttribute()`, and `removeAttribute()`.
- To attach event handlers so that each modification is triggered by a user action.

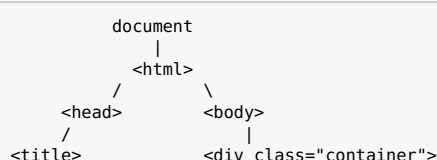
4. THEORY

1. What is the DOM?

The Document Object Model (DOM) is a programming interface for HTML and XML documents. When a browser loads a webpage, it parses the HTML source and builds an in-memory tree structure in which every element, attribute, and piece of text becomes an **object (node)**. JavaScript can then read and modify these objects, and the browser immediately re-renders the affected part of the page. The DOM is therefore the bridge between the static HTML document and dynamic scripting.

2. Structure of the DOM Tree

The DOM is organised as a hierarchical tree. The document object is the root, below which lie the `<html>` element, then `<head>` and `<body>`, and then all nested elements and their text nodes.





3. Types of Nodes

- **Document node** - the root document object representing the whole page.
- **Element node** - every HTML tag such as `<div>` or `<p>`.
- **Attribute node** - attributes such as `id`, `class`, `src`.
- **Text node** - the actual text written inside an element.
- **Comment node** - HTML comments present in the source.

4. DOM Selection Methods

- `document.getElementById('id')` - returns the single element with the given unique id.
- `document.getElementsByClassName('cls')` - returns a live `HTMLCollection` of elements.
- `document.getElementsByTagName('p')` - returns a live `HTMLCollection` of that tag.
- `document.querySelector('.cls')` - returns the first element matching a CSS selector.
- `document.querySelectorAll('.cls')` - returns a static `NodeList` of all matches.

5. Changing Content

- `element.textContent` - gets or sets the plain text of an element. HTML tags inside the assigned string are treated as literal text, which makes it safe against cross-site scripting.
- `element.innerHTML` - gets or sets the markup inside an element. The assigned string is parsed as HTML, so tags are rendered.
- `element.innerText` - similar to `textContent` but it respects CSS visibility.
- `document.createElement()` with `appendChild()` or `removeChild()` - creates and inserts or deletes actual nodes in the DOM tree.

6. Changing Style

- `element.style.property` - applies an inline CSS declaration. CSS property names written with a hyphen are converted to camelCase, for example `background-color` becomes `backgroundColor`.
- `element.classList.add()` / `remove()` / `toggle()` / `contains()` - adds or removes predefined CSS classes. This is the preferred approach because presentation logic stays in the stylesheet.
- Inline styles written through the `style` property have a higher specificity than the rules defined in the external stylesheet.

7. Changing Attributes

- `element.getAttribute('name')` - returns the current value of an attribute, or `null` when the attribute is absent.
- `element.setAttribute('name', 'value')` - modifies an existing attribute or creates a new one.
- `element.removeAttribute('name')` - deletes an attribute from the element.
- `element.hasAttribute('name')` - returns a boolean.
- Custom `data-*` attributes can also be read through the `dataset` property.

8. Events and the DOM

DOM modification is normally triggered by user interaction. Handlers can be attached through an inline HTML attribute such as `onclick`, or, in the recommended approach, through `element.addEventListener('click', handler)`, which separates behaviour from structure and allows several handlers on the same element.

5. ALGORITHM / PROCEDURE

1. Start.
2. Create `index.html` with a target paragraph, an image, an anchor element, an unordered list, and separate button groups for content, style, and attribute operations.
3. Create `style.css` and define the base card layout together with a `.highlight` class that will be applied dynamically.
4. Create `script.js` and obtain references to the target elements using `document.getElementById()`.
5. Store the original text, image source, and link href in constants so that the page can be restored later.
6. Define `changeText()` to replace the paragraph text using `textContent`.
7. Define `changeHTML()` to insert formatted markup using `innerHTML`.
8. Define `addListItem()` to create a new `<li>` node with `createElement()` and attach it with `appendChild()`.
9. Define `changeStyle()` to set colour, background, font size, and padding through the style property.
0. Define `toggleHighlight()` to add or remove the CSS class using `classList.toggle()`.
1. Define `changeAttributes()`, `readAttribute()`, and `removeAttributeDemo()` to set, read, and delete attributes.
2. Define `resetAll()` to restore the original content, styles, and attributes.
3. Bind each function to the corresponding button through the `onclick` attribute.
4. Open the page in a browser, click each button, and observe the changes taking place instantly without a page reload.
5. Stop.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DOM Manipulation Demonstration</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <header class="header">
      <h1>Document Object Model (DOM) Demonstration</h1>
      <p class="subtitle">Full Stack Web Development (23CM4121) - CO-1 Task Demonstration</p>
    </header>

    <!-- ===== TARGET ELEMENT ON WHICH ALL OPERATIONS ARE PERFORMED ===== -->
    <section class="card">
      <h2>Target Element</h2>
      <p id="demoText" class="demo-text" title="Original title attribute">
        This is the original paragraph text loaded from the HTML file.
      </p>
      
      <a id="demoLink" href="https://www.w3.org" target="_blank">Visit W3C</a>
    </section>

    <!-- ===== 1. CHANGING CONTENT ===== -->
    <section class="card">
      <h2>1. Changing Content</h2>
      <button class="btn" onclick="changeText()">Change textContent</button>
      <button class="btn" onclick="changeHTML()">Change innerHTML</button>
      <button class="btn" onclick="addListItem()">Append New Node</button>
      <ul id="itemList">
        <li>Existing Item 1</li>
        <li>Existing Item 2</li>
      </ul>
    </section>

    <!-- ===== 2. CHANGING STYLE ===== -->
    <section class="card">
      <h2>2. Changing Style</h2>
      <button class="btn" onclick="changeStyle()">Apply Inline Style</button>
      <button class="btn" onclick="toggleHighlight()">Toggle CSS Class</button>
    </section>

    <!-- ===== 3. CHANGING ATTRIBUTES ===== -->
    <section class="card">
      <h2>3. Changing Attributes</h2>
      <button class="btn" onclick="changeAttributes()">Change Image and Link</button>
      <button class="btn" onclick="readAttribute()">Read Attribute Value</button>
      <button class="btn" onclick="removeAttributeDemo()">Remove Attribute</button>
      <p id="attrOutput" class="output"></p>
    </section>

    <!-- ===== RESET ===== -->
    <section class="card">
      <button class="btn btn-reset" onclick="resetAll()">Reset Everything</button>
    </section>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

style.css

```
* { margin: 0; padding: 0; box-sizing: border-box;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; }

body { background: #f0f4f8; color: #2d3748; padding: 30px 20px; }

.container { max-width: 820px; margin: 0 auto; }

.header { text-align: center; margin-bottom: 26px; }
.header h1 { color: #1a365d; font-size: 26px; margin-bottom: 6px; }
.subtitle { color: #718096; font-size: 14px; }

.card { background: #ffffff; border: 1px solid #e2e8f0; border-radius: 12px;
  padding: 20px 24px; margin-bottom: 20px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.06); }

.card h2 { font-size: 18px; color: #2b6cb0; margin-bottom: 14px;
  border-bottom: 2px solid #ebf8ff; padding-bottom: 6px; }

.demo-text { font-size: 16px; line-height: 1.6; padding: 12px;
  border: 1px dashed #cbd5e0; border-radius: 6px; margin-bottom: 12px; }

/* Class applied and removed dynamically through classList */
.highlight { background-color: #fefcbf; color: #744210;
  border-left: 5px solid #d69e2e; font-weight: 600; }

.btn { background: #3182ce; color: #ffffff; border: none; padding: 9px 16px;
  margin: 4px 6px 4px 0; border-radius: 6px; font-size: 14px;
  cursor: pointer; transition: background 0.2s; }
```

```
.btn:hover { background: #2b6cb0; }
.btn-reset { background: #e53e3e; width: 100%; }
.btn-reset:hover { background: #c53030; }

#itemList { margin: 12px 0 0 22px; }
#itemList li { margin: 4px 0; }

.output { margin-top: 12px; font-size: 14px; color: #2b6cb0;
          background: #ebf8ff; padding: 10px 12px; border-radius: 6px;
          min-height: 20px; }

a { display: inline-block; margin-top: 10px; color: #2b6cb0; }
```

script.js

```
/**
 * script.js
 * Demonstrates how JavaScript dynamically changes the CONTENT, STYLE and
 * ATTRIBUTES of HTML elements through the Document Object Model (DOM).
 */

// ----- DOM element references (selection) -----
const demoText = document.getElementById('demoText');
const demoImage = document.getElementById('demoImage');
const demoLink = document.getElementById('demoLink');
const itemList = document.getElementById('itemList');
const attrOutput = document.getElementById('attrOutput');

// Store the original state so that the page can be reset
const ORIGINAL_TEXT = demoText.textContent;
const ORIGINAL_HTML = demoText.innerHTML;
const ORIGINAL_SRC = demoImage.getAttribute('src');
const ORIGINAL_HREF = demoLink.getAttribute('href');

let itemCount = 2; // Counter for dynamically created list items

/* =====
 1. CHANGING THE CONTENT OF AN ELEMENT
 ===== */

// textContent replaces only the plain text of the element
function changeText() {
    demoText.textContent = 'The content of this paragraph was changed using textContent.';
}

// innerHTML parses the assigned string as HTML markup
function changeHTML() {
    demoText.innerHTML =
        'The content was changed using <b>innerHTML</b>, so this text is <i>formatted</i>.';
}

// Creating and appending a brand-new element node to the DOM tree
function addListItem() {
    itemCount++;
    const li = document.createElement('li'); // create a new node
    li.textContent = 'Dynamically Created Item ' + itemCount; // set its text
    itemList.appendChild(li); // attach it to the DOM
}

/* =====
 2. CHANGING THE STYLE OF AN ELEMENT
 ===== */

// Inline style through the style property (camelCase property names)
function changeStyle() {
    demoText.style.color = '#ffffff';
    demoText.style.backgroundColor = '#2b6cb0';
    demoText.style.fontSize = '18px';
    demoText.style.padding = '16px';
    demoText.style.borderRadius = '8px';
}

// Adding / removing a CSS class through classList (preferred approach)
function toggleHighlight() {
    demoText.classList.toggle('highlight');
}

/* =====
 3. CHANGING THE ATTRIBUTES OF AN ELEMENT
 ===== */

// setAttribute() modifies existing attributes or creates new ones
function changeAttributes() {
    demoImage.setAttribute('src', 'images/javascript.png');
    demoImage.setAttribute('alt', 'JavaScript Logo');
    demoImage.setAttribute('width', '160');

    demoLink.setAttribute('href', 'https://developer.mozilla.org');
    demoLink.textContent = 'Visit MDN Web Docs';

    attrOutput.textContent = 'Image source and link href were changed using setAttribute().';
}

// getAttribute() reads the current value of an attribute
function readAttribute() {
    const src = demoImage.getAttribute('src');
    const href = demoLink.getAttribute('href');
    const title = demoText.getAttribute('title');
```

```

    attrOutput.textContent =
        'src = ' + src + ' | href = ' + href + ' | title = ' + title;
}

// removeAttribute() deletes an attribute from the element
function removeAttributeDemo() {
    demoText.removeAttribute('title');
    attrOutput.textContent =
        'The title attribute was removed. getAttribute("title") now returns : ' +
        demoText.getAttribute('title');    // returns null
}

/* =====
   RESET THE PAGE BACK TO ITS ORIGINAL STATE
   ===== */
function resetAll() {
    demoText.innerHTML = ORIGINAL_HTML;
    demoText.removeAttribute('style');    // clears every inline style
    demoText.classList.remove('highlight');
    demoText.setAttribute('title', 'Original title attribute');

    demoImage.setAttribute('src', ORIGINAL_SRC);
    demoImage.setAttribute('alt', 'HTML5 Logo');
    demoImage.setAttribute('width', '120');

    demoLink.setAttribute('href', ORIGINAL_HREF);
    demoLink.textContent = 'Visit W3C';

    itemList.innerHTML = '<li>Existing Item 1</li><li>Existing Item 2</li>';
    itemCount = 2;

    attrOutput.textContent = 'Page has been reset to its original state.';
}

```

7. CODE EXPLANATION

Code	Explanation
<code>document.getElementById('demoText')</code>	Selects the target paragraph node from the DOM tree using its unique id.
<code>element.textContent = '...'</code>	Replaces only the plain text of the element; any markup in the string is shown literally.
<code>element.innerHTML = '...'</code>	Replaces the inner markup of the element; the assigned string is parsed as HTML.
<code>document.createElement('li')</code>	Creates a brand-new element node that is not yet part of the document.
<code>parent.appendChild(li)</code>	Attaches the newly created node as the last child of the parent element.
<code>element.style.color</code>	Applies an inline CSS declaration directly to the element.
<code>element.style.backgroundColor</code>	Camel-case form of the CSS property background-color, as required by JavaScript.
<code>classList.toggle('highlight')</code>	Adds the CSS class if it is absent and removes it if it is already present.
<code>element.setAttribute(name, value)</code>	Creates a new attribute or modifies the value of an existing attribute.
<code>element.getAttribute(name)</code>	Returns the current value of the attribute, or null if it does not exist.
<code>element.removeAttribute('title')</code>	Deletes the specified attribute from the element completely.
<code>removeAttribute('style')</code>	Clears every inline style that was applied earlier, restoring the stylesheet rules.
<code>onclick="changeText()"</code>	Inline event binding that invokes the handler function when the button is clicked.
<code>const ORIGINAL_HTML = ...</code>	Stores the initial markup at load time so that the reset function can restore it.

8. TEST CASES AND EXECUTION DATA

The following test cases were recorded after executing the webpage in a browser.

Test Case	TC-01
Test / Input	Click the “Change textContent” button.
Expected Result	The paragraph text is replaced with the new sentence instantly, and no HTML formatting is applied.
Actual Result	Text updated immediately without a page reload, confirming that textContent modifies the text node only.
Status	Pass

TC-01 Output:

Document Object Model (DOM) Demonstration

Full Stack Web Development (23CM4121) - CO-1 Task Demonstration

Target Element

The content of this paragraph was changed using `textContent`.



[Visit W3C](#)

1. Changing Content

Change `textContent`

Change `innerHTML`

Append New Node

- Existing Item 1
- Existing Item 2

2. Changing Style

Apply Inline Style

Toggle CSS Class

3. Changing Attributes

Change Image and Link

Read Attribute Value

Remove Attribute

Reset Everything

Test Case	TC-02
Test / Input	Click the “Change innerHTML” button and then the “Append New Node” button.
Expected Result	The paragraph shows bold and italic formatted text, and a new list item “Dynamically Created Item 3” is added to the list.
Actual Result	The markup was rendered correctly and the new node was appended to the DOM tree as expected.
Status	Pass

TC-02 Output:

Document Object Model (DOM) Demonstration

Full Stack Web Development (23CM4121) - CO-1 Task Demonstration

Target Element

The content was changed using **innerHTML**, so this text is *formatted*.



[Visit W3C](#)

1. Changing Content

Change.textContent

Change.innerHTML

Append New Node

- Existing Item 1
- Existing Item 2
- Dynamically Created Item 3

2. Changing Style

Apply Inline Style

Toggle CSS Class

3. Changing Attributes

Change Image and Link

Read Attribute Value

Remove Attribute

Reset Everything

Test Case	TC-03
Test / Input	Click the “Apply Inline Style” button and then the “Toggle CSS Class” button twice.
Expected Result	The paragraph turns white on a blue background with a larger font, and the yellow highlight class is added and then removed.
Actual Result	Inline styles were applied and the CSS class was toggled on and off correctly.
Status	Pass

TC-03 Output:

Document Object Model (DOM) Demonstration

Full Stack Web Development (23CM4121) - CO-1 Task Demonstration

Target Element

This is the original paragraph text loaded from the HTML file.



[Visit W3C](#)

1. Changing Content

Change textContent

Change innerHTML

Append New Node

- Existing Item 1
- Existing Item 2

2. Changing Style

Apply Inline Style

Toggle CSS Class

3. Changing Attributes

Change Image and Link

Read Attribute Value

Remove Attribute

Reset Everything

Test Case	TC-04
Test / Input	Click “Change Image and Link” and then “Read Attribute Value”.
Expected Result	The image source changes to javascript.png with width 160, the link points to MDN, and the output line displays the current src, href, and title values.
Actual Result	Attributes were modified with setAttribute() and the new values were correctly read back with getAttribute().
Status	Pass

TC-04 Output:

Document Object Model (DOM) Demonstration

Full Stack Web Development (23CM4121) - CO-1 Task Demonstration

Target Element

This is the original paragraph text loaded from the HTML file.



[Visit MDN Web Docs](#)

1. Changing Content

Change.textContent

Change.innerHTML

Append New Node

- Existing Item 1
- Existing Item 2

2. Changing Style

Apply Inline Style

Toggle CSS Class

3. Changing Attributes

Change Image and Link

Read Attribute Value

Remove Attribute

src = images/javascript.png | href = https://developer.mozilla.org | title = Original title attribute

Reset Everything

Test Case	TC-05
Test / Input	Click “Remove Attribute” and then “Reset Everything”.
Expected Result	The title attribute is deleted and getAttribute(“title”) returns null; the reset button restores the original text, styles, and attributes.
Actual Result	The attribute was removed and null was displayed, and the page was fully restored to its initial state.
Status	Pass

TC-05 Output:

Document Object Model (DOM) Demonstration

Full Stack Web Development (23CM4121) - CO-1 Task Demonstration

Target Element

This is the original paragraph text loaded from the HTML file.



[Visit W3C](#)

1. Changing Content

Change.textContent

Change.innerHTML

Append New Node

- Existing Item 1
- Existing Item 2

2. Changing Style

Apply Inline Style

Toggle CSS Class

3. Changing Attributes

Change Image and Link

Read Attribute Value

Remove Attribute

The title attribute was removed. `getAttribute("title")` now returns : null

Reset Everything

9. OUTPUT

Output 1: Webpage after Dynamic Content, Style, and Attribute Manipulation

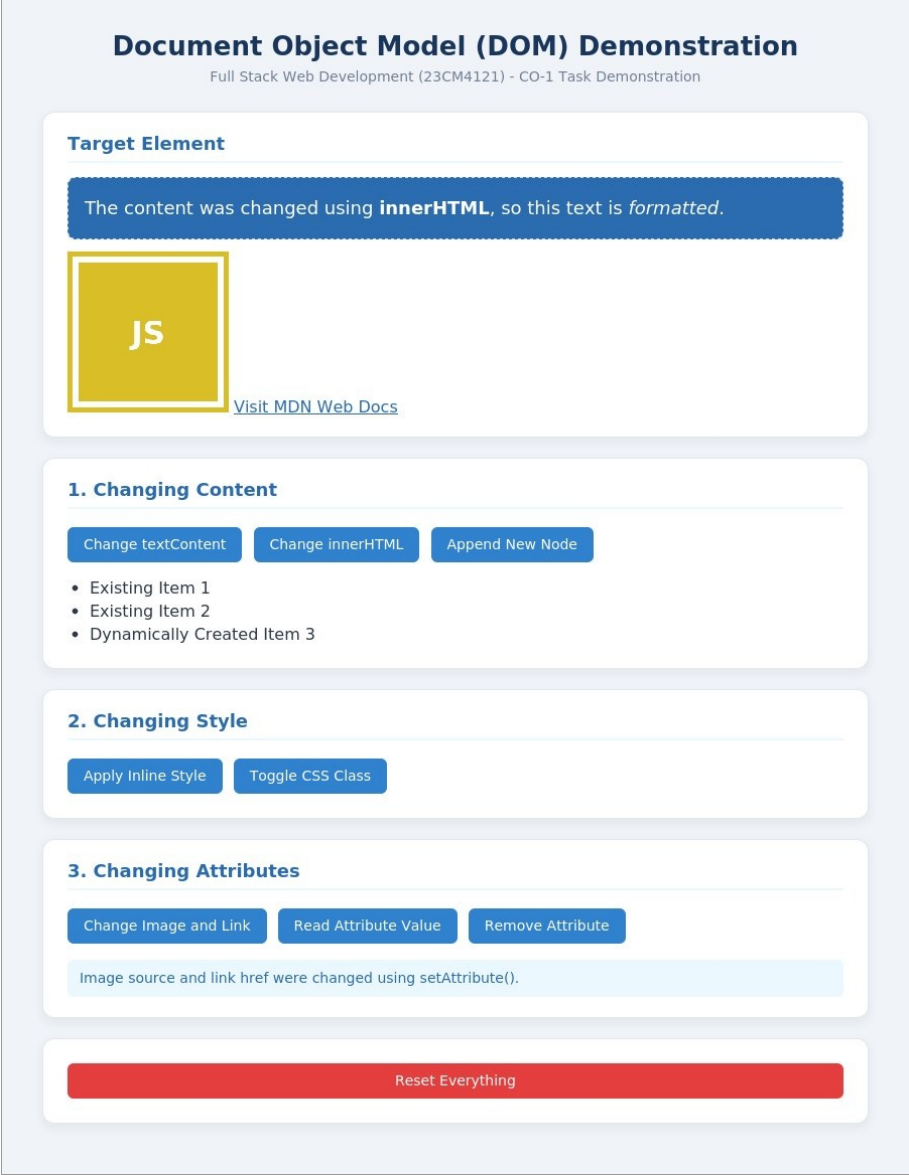


Figure 1: Full Webpage Output of the DOM Manipulation Demonstration

10. RESULT

Thus, the **Document Object Model** was studied and a webpage was successfully developed to demonstrate dynamic modification of the content, style, and attributes of HTML elements using JavaScript. All changes were reflected instantly in the browser without reloading the page, confirming the working of DOM selection methods, `textContent/innerHTML`, the `style` property, the `classList` API, and the attribute manipulation methods.